

SMART CAMPUS EVENT MANAGMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering
By**

PARI ABHISHEK (VTU32463)

*10211CS224
Full Stack Application Development
Slot : E3-B9*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "SMART CAMPUS EVENT MANAGEMENT SYSTEM" by PARI ABHISHEK (VTU32463) has been carried out in Winter semester of Academic year 2025 -2026 (WS2526).

Signature of Faculty

Dr.Hafizur Rahaman.

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr.Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(PARI ABHISHEK)

Date:06/05/2026

ABSTRACT

The Smart Campus Event Management System is a full-stack web application that automates and simplifies event management in colleges and universities. It allows students to view and register for events like workshops and seminars, while administrators can create, update, delete, and manage events using filters. Built with Spring Boot (backend) and React JS (frontend), it provides a responsive and user-friendly interface, improving efficiency, reducing manual work, and enhancing communication within the campus.

Keywords: Smart Campus, Event Management System, Full-Stack Web Application, Spring Boot, React JS, RESTful APIs, Spring Data JPA, CRUD Operations, Student Registration, Admin Dashboard, Event Scheduling, Data Validation, Authentication, Database Management, Web Application Development

LIST OF FIGURES

1.1	Framework	2
3.1	Backend Implementation	10
4.1	Project Architecture	13
4.2	Data Flow Diagram	14
5.1	Landing Page Interface	18
5.2	Authentication Page	18
5.3	User Dashboard	19
5.4	Admin Dashboard	20
5.5	Events Page	20
5.6	Registered Events	21
5.7	Profile Page	22
5.8	Signout Page	22

LIST OF TABLES

2.1	Comparative Analysis of Campus Event Management Systems	4
3.1	System Technology Stack	12
5.1	Event Management System Module Comparison . . .	23
7.1	Mapping of Project to Complex Engineering Problem (CEP)	26
7.2	Mapping of Project to Complex Engineering Problem (CEP)	27
7.3	SDG Mapping of the Project	28

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Casacading Style Sheets
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JS	JavaScript
JSON	JavaScript Object Notation
MVC	Model View Controller
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
UX	User Experience

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	2
1.4 Framework	2
2 SURVEY OF SIMILAR APPLICATION IN MARKET	3
3 FRAMEWORK DESCRIPTION	5
3.1 Frontend Implementation	5

3.2	Backend Implementation	7
3.3	Database Implementation	10
3.4	System Technology Stack Overview	12
4	METHODOLOGIES	13
4.1	Project Architecture	13
4.2	DataFlow Diagram	14
4.3	Module 1:	15
4.4	Module 2:	15
4.5	Module 3:	15
5	RESULTS AND DISCUSSIONS	17
5.1	Results and Discussion	17
6	CONCLUSION AND FUTURE ENHANCEMENTS	24
6.1	Conclusion	24
6.2	Future Enhancements	24
7	Addressing Complex Engineering Problem and SDG	26
7.1	Mapping of the Project to Complex Engineering Problem (CEP)	26
7.2	Mapping of the Project to Complex Engineering Activities (CEA)	27
7.3	Sustainable Development Goals (SDG) Mapping . . .	28

8	REFERENCES	29
----------	-------------------	-----------

	References	29
--	-------------------	-----------

Chapter 1

INTRODUCTION

1.1 Introduction

The Smart Campus Event Management System is a web-based application that simplifies the organization of campus events by providing a centralized platform for students to view and register for events, while administrators can manage event details efficiently. Built using Spring Boot and React JS, it ensures secure, efficient, and streamlined event management, reducing manual effort and improving coordination.

1.2 Aim of the Project

To develop a web-based system that simplifies campus event management by enabling easy event registration for students and efficient event handling for administrators. To create a centralized platform that automates event organization, improves communication.

1.3 Scope of the Project

The system enables students to view, register, and give feedback for events, while administrators can create, update, delete, and manage events efficiently through a web-based platform. It includes features like event search and filtering, registration tracking, and secure access.

1.4 Framework

Shows the system framework consists of layers like user interface, application logic, and database.

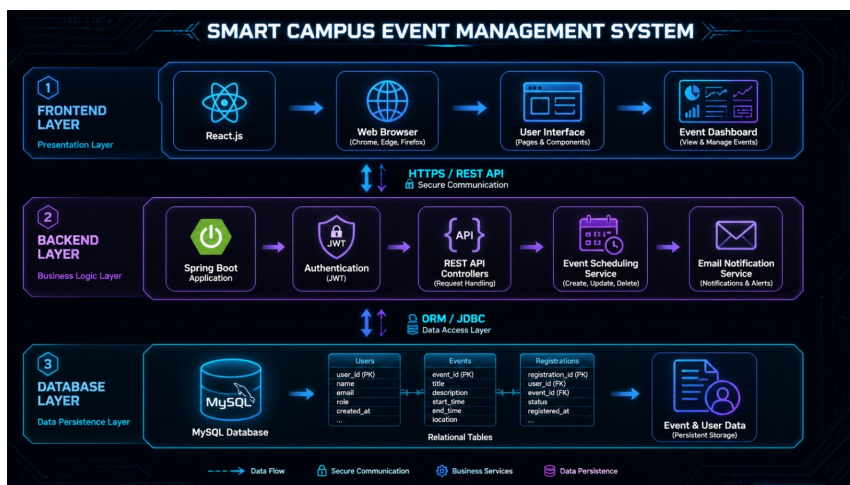


Figure 1.1: Framework

Figure 1.1 Describes the Smart Campus Event Management System framework is designed with a layered architecture integrating a React-based frontend, a Spring Boot backend, and a relational database for efficient data handling.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

A Smart Campus Event Management System is designed for colleges to manage and organize events efficiently. It is developed using React.js [1] with Bootstrap [2] for a responsive and user-friendly frontend, while Node.js [8] with Express.js [9] is used to build a scalable and efficient backend API layer. The system includes features like event creation, student registration, real-time updates, and digital tickets.

For testing and API validation, Postman [7] is used. Data is stored and managed using MySQL [3], ensuring structured and reliable database handling, while Firebase [5] is used for authentication, notifications, and real-time updates. The system supports admins and students with role-based access, notifications, and feedback, making event management simple, organized, and user-friendly. .

Additionally, such systems focus on operational efficiency by offering real-time registration updates, automated workflows, and centralized dashboards. This reduces manual work for administrators and improves overall coordination between departments.

Unlike large-scale public event platforms, a campus-focused system is designed to handle internal institutional needs. It supports restricted access for specific students or departments, simplified registration processes, and direct communication between organizers and participants. This makes it more suitable for managing academic, technical, and cultural events within a college environment.

Table 2.1: Comparative Analysis of Campus Event Management Systems

Feature	Smart Campus Event Management System
Event Creation	Yes(Admin/Faculty can create events easily)
Event Categories	Yes(Workshops,Seminars,Clubs,Cultural,Technical)
Student Registration	Yes(Simple form with validation)
User Login System	Yes(Student/Admin role-based login)
Event Approval Workflow	Yes(Admin approval before publishing events)
Notifications	Yes(Email/Dashboard alerts)
Event Analytics	Yes(Basic reports like participation count)
Multi-User Roles	Yes(Admin,Student,Organizer)

Table 2.1: compares the system allows easy event creation, categorization, student registration, and secure role-based login with admin approval before publishing. It also provides notifications, analytics, and supports multiple user roles like Admin, Student, and Organizer.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend for the Smart Campus Event Management System is developed using modern web technologies such as HTML,MySQL CSS,JavaScript, and React.js. Development tools like Visual Studio Code and web browsers (Chrome/Edge) are used for coding and testing. Required libraries such as React Router for navigation, Axios for API communication, and Bootstrap or Tailwind CSS for styling are installed to enhance UI design and responsiveness.

3.1.2 Structure of the Project

The project follows a modular and scalable structure to maintain clarity and ensure easy development:

/public: Contains static files such as index.html, campus logos, event posters, and images used across the application.

/src/api: Includes configuration files (e.g., api.js or axios.js) to manage communication with the backend server for fetching and storing event data.

/src/components: Contains reusable UI components such as Navbar, Footer, EventCard, RegistrationForm, NotificationPanel, and QR Code generator for event entry.

/src/context: Handles global state management using Context API (e.g., AuthContext.js, EventContext.js) for managing user sessions, roles (Admin/Student), and event data.

/src/pages: Includes the main pages of the application-
Home.js: Displays upcoming campus events and announcements.
EventDetails.js: Shows complete details of selected events including date, venue, and description.

RegisterEvent.js: Handles student registration for events with validation.

AdminDashboard.js: Provides admin controls to create, update, or delete events and manage participants.

MyEvents.js: Allows students to view their registered events and participation status.

/src/utlis: Contains helper functions such as form validation, date/time formatting, and notification handling.

App.js: The main component that manages routing, navigation, and

overall layout of the Smart Campus Event Management System.

3.1.3 Execution Procedure

The project is executed by opening the main HTML file in a web browser or running it through a local development server. Users can interact with the interface to view events, register events and perform other actions, while developers can test and debug features during execution.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend environment for the Smart Campus Event Management System is configured with essential tools and technologies to support smooth development and execution:

Frontend Layer (React Application): Node.js (v14 or above) is used to run the development server and manage dependencies. npm or Yarn is used for package management. React.js is used to build an interactive and responsive user interface for students and administrators.

Backend Layer (Spring Boot): Java Development Kit (JDK 8 or above) is required for application development. Apache Maven is used for dependency management and build automation. Spring Boot framework is used to develop RESTful APIs and implement business

logic such as event creation, registration, and user management.

Database Layer: MySQL (v8.0 or above) is used to store data such as user details, event information, registrations, and feedback. MySQL Workbench (optional) can be used for database design and administration.

Integration Tools: Axios is used in the frontend to communicate with backend APIs. Postman is used for testing API endpoints. Git is used for version control and collaboration during development.

Deployment Environment (Optional): The system can be run on a localhost server for development and testing. It can also be deployed on cloud platforms for real-time access within the campus.

3.2.2 Structure of the Project

The backend project follows a layered architecture to ensure clean code organization and scalability:

/controller: Contains classes that handle HTTP requests and responses for events, users, and registrations.

/service: Includes business logic such as event scheduling, student registration, and notification handling.

/repository: Manages database operations using JPA repositories for entities like User, Event, and Registration.

/model: Defines entity classes that represent database tables.

/config: Contains configuration files for database connection, secu-

rity, and application settings.

/security: Handles authentication and authorization (e.g., role-based access for Admin and Student).

application.properties: Stores configuration details such as database credentials and server settings.

3.2.3 Execution Procedure

The backend application is executed using Spring Boot. The server is started using commands like `mvn spring-boot:run` or by running the main application class. Once started, the backend listens for client requests from the frontend, processes event-related operations, and interacts with the database.

APIs are tested using tools like Postman to ensure proper functionality. After successful execution, the system allows users to register for events, and administrators to manage campus activities efficiently. The system maintains logs for important activities such as user login, event creation, and registration. These logs help in monitoring system performance and debugging issues when they occur. Logging frameworks can be used to track system behavior in real time.

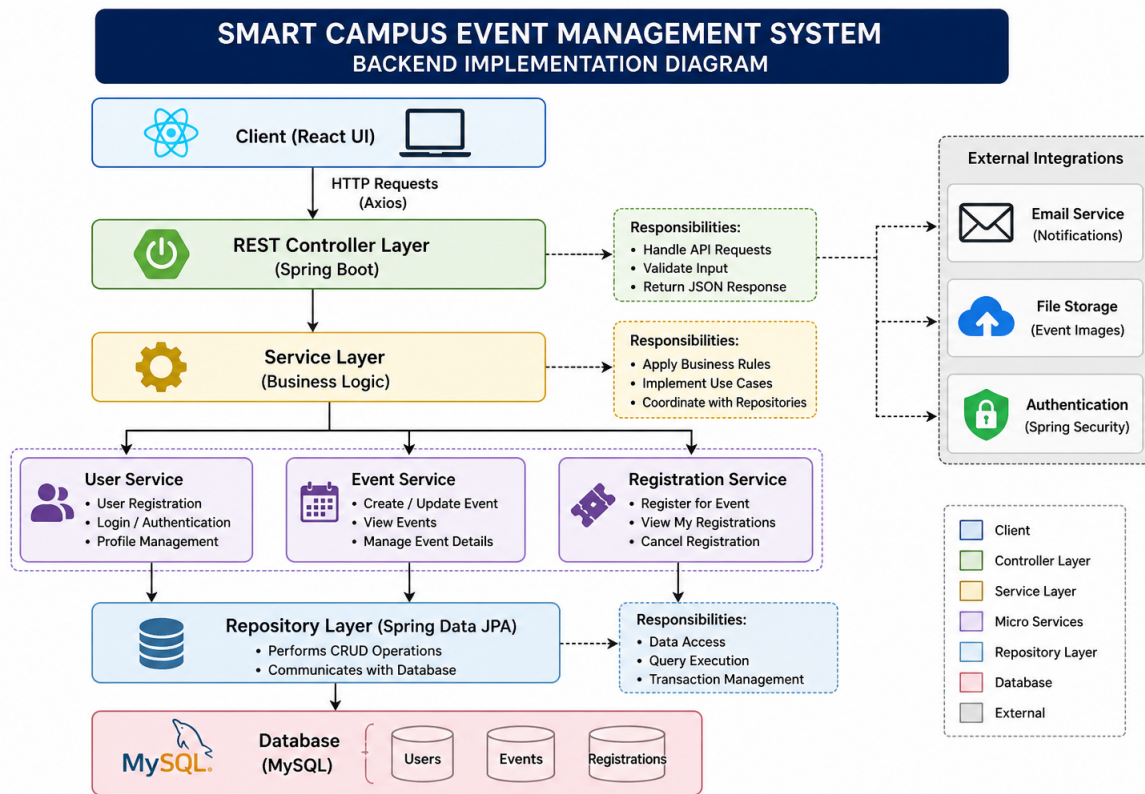


Figure 3.1: Backend Implementation

Figure 3.1: Describes the backend of the Smart Campus Event Management System is developed using Node.js and Express.js, which provide a lightweight, scalable, and high-performance server-side environment.

3.3 Database Implementation

3.3.1 Database Configuration

The database for the Smart Campus Event Management System is configured using a relational database management system such as MySQL. Proper setup includes database creation, user access control,

and secure connection with the backend using JDBC. The system ensures storage and retrieval of data related to students, events, registrations, and feedback while maintaining data security and integrity.

3.3.2 Schema Structure

The database schema is designed with well-structured tables and relationships to support campus event operations. It includes entities such as users, events, registrations, and feedback, connected using primary and foreign keys. This design ensures data consistency, avoids redundancy, and enables efficient querying.

3.3.3 Tables Created

The main functionalities of the system are supported by the following tables:

Users: Stores user details such as user ID, name, email, encrypted password, role (Admin/Student), and contact information. It is used for authentication and role-based access control.

Events: Contains information about campus events including event ID, event name, description, date, time, venue, department, and organizer details.

Registrations: Acts as a link between users and events. It stores registration details such as registration ID, user ID, event ID, registration date, and participation status.

Feedback: Stores feedback provided by students after attending events, including feedback ID, user ID, event ID, ratings, and comments.

Notifications: Maintains records of event updates, announcements sent to users, including notification ID, message content, and timestamp.

3.4 System Technology Stack Overview

The following table provides a comprehensive summary of the frameworks and technologies used in the development of the Smart Campus Event Management System, categorized based on their architectural roles.

Table 3.1: System Technology Stack

Component	Technology
Frontend	React.js, HTML5, CSS3
Backend	Spring Boot, Spring Security
Database	MySQL
API Communication	Axios
Tools	Git, Postman, VS Code

Table 3.1: The system uses React for the frontend and Spring Boot for backend processing, with MySQL for data storage and Axios for API communication.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The Smart Campus Event Management System uses a layered architecture with a React JS frontend, Spring Boot backend, and a relational database. This design ensures scalability, security, efficient performance, and smooth communication between all components.

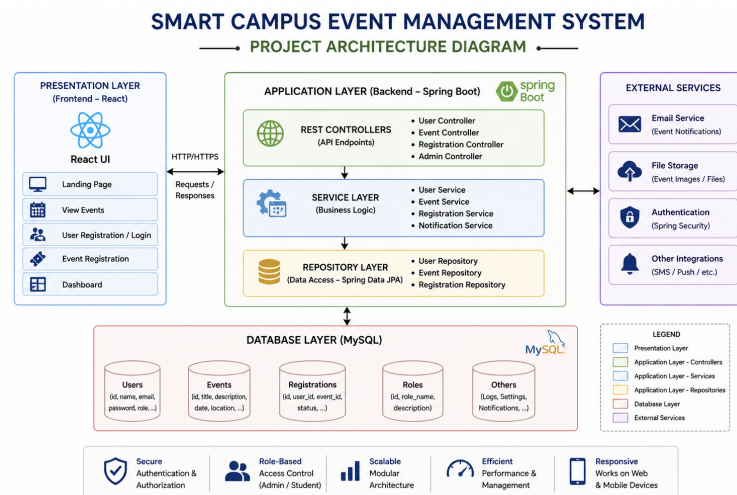


Figure 4.1: Project Architecture

Figure 4.1: Describes the Smart Campus Event Management System

follows a client–server architecture with a clear separation between the frontend, backend, and database layers.

4.2 DataFlow Diagram

The Data Flow Diagram (DFD) shows how user data flows from the frontend to the backend, gets processed and stored in the database, and returns as outputs like event details or registration confirmation, ensuring smooth system. The system processes data using authentication, event management, registration, notifications, and feedback modules. All information is stored and managed in a MySQL database with real-time support through Firebase services communication.

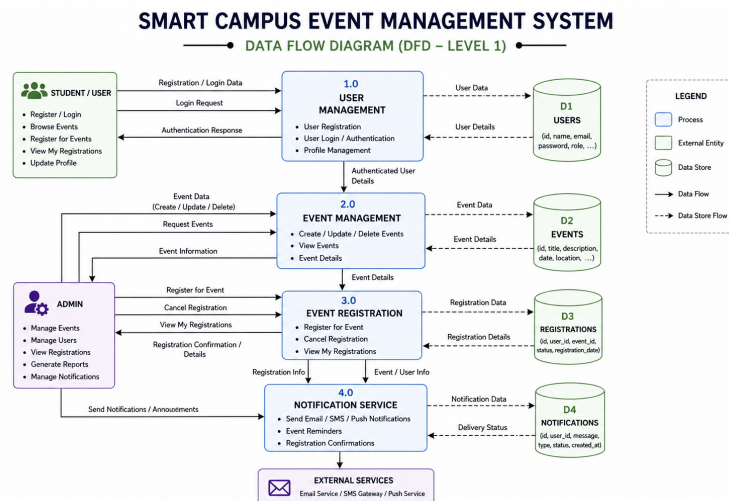


Figure 4.2: Data Flow Diagram

Figure 4.2 Describes the data flow in the Smart Campus Event Management System describes how information moves between users, and the database to ensure smooth event management operations.

4.3 Module 1:

User Management: This module handles user registration, login, and authentication. It ensures that only authorized users can access the system and perform registration operations.

4.4 Module 2:

Event Management: This module enables administrators to create, update, and manage campus event details such as date, time, venue, and participant capacity, ensuring proper organization and scheduling of events.

4.5 Module 3:

Event Registration: This module enables students to register for campus events, check seat or slot availability, and receive confirmation. It also prevents over-registration through proper validation and capacity control.

Advantages of this Project: The system simplifies event registration, reduces manual effort, and improves overall efficiency. It ensures accurate data management and provides a user-friendly interface for both students and administrators.

a) Time Saving: Automates the event registration process, reducing manual effort and saving time.

b) Error Reduction: Minimizes human errors and prevents over-registration through proper validation.

c) Easy Management: Helps administrators efficiently manage events, participants, and schedules.

Disadvantages: The system relies on internet connectivity and may face technical issues if not properly maintained. Initial setup and development require time and technical expertise.

Chapter 5

RESULTS AND DISCUSSIONS

5.1 Results and Discussion

Results: The Smart Campus Event Management System was successfully developed and tested. The system effectively handled user registration, event browsing, event registration, and feedback submission. Features such as event creation, search and filtering, and admin management worked as expected. The admin dashboard allowed efficient control over events and user activities. Overall, the system provided a smooth, user-friendly experience with accurate data handling and reliable performance.

Screenshots:

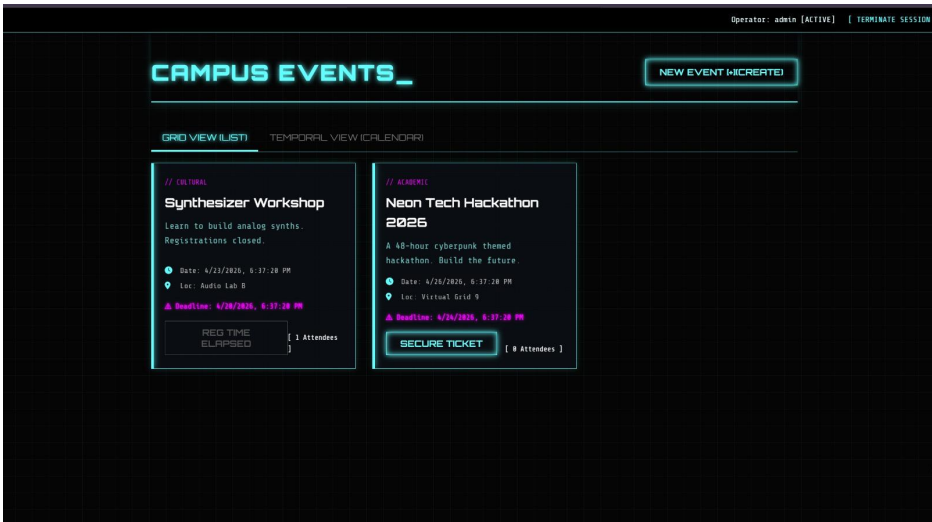


Figure 5.1: Landing Page Interface

Figure 5.1: The landing page is the first interaction point for users of the Smart Campus Event Management System. It provides a simple overview of the platform and easy navigation to key features like event browsing and login. Built using React, it ensures a responsive, fast, and user-friendly interface across all devices.



Figure 5.2: Authentication Page

Figure 5.2 Describes the User Authentication module is responsible for managing secure access to the Smart Campus Event Management

System. It ensures that only authorized users can access system features based on their roles. The module is implemented using Spring Boot for backend services and integrates with the frontend built using React.

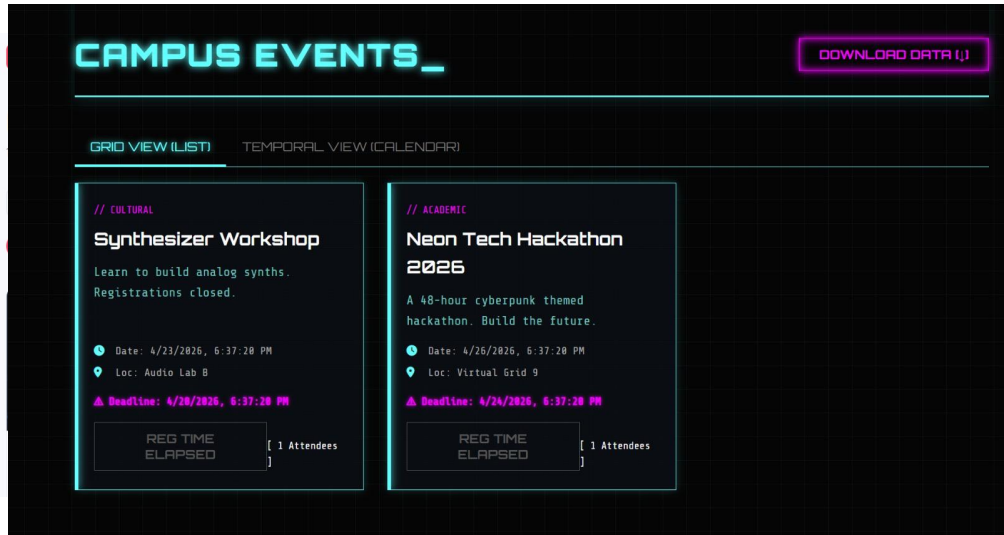


Figure 5.3: User Dashboard

Figure 5.3 Describes the User Dashboard is a personalized interface that allows users to manage their activities within the Smart Campus Event Management System. It provides a centralized view of all relevant information, enabling users to easily access events, registrations, and profile details. The dashboard is developed using React for a dynamic and responsive user experience.

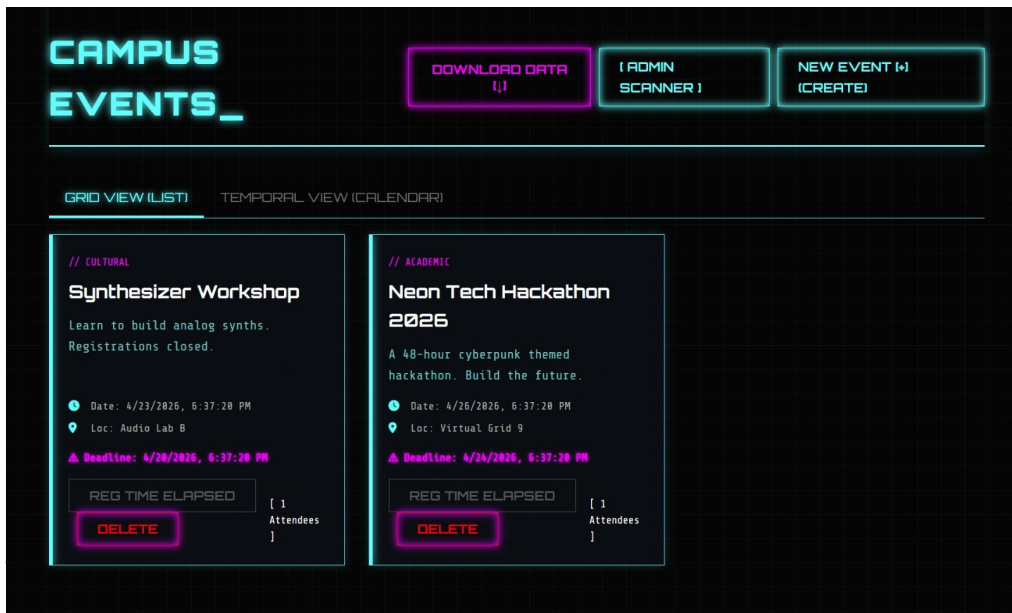


Figure 5.4: Admin Dashboard

Figure 5.4: Describes the Admin Dashboard is a centralized interface designed for administrators to manage all campus events and user activities efficiently. It provides complete control over event creation, user management, and system monitoring. The dashboard is developed using React and communicates with the backend powered by Spring Boot.

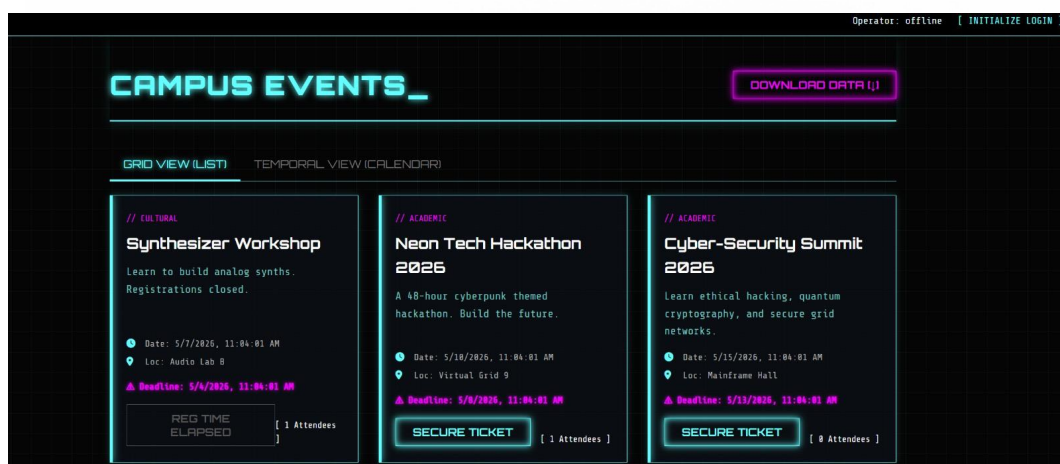


Figure 5.5: Events Page

Figure 5.5: Describes the Events module is a core component of the

Smart Campus Event Management System that enables the creation, management, and display of campus events. It provides a centralized platform where users can explore upcoming events and administrators can organize and manage them efficiently. The module is developed using React for the user interface and powered by Spring Boot for backend processing.

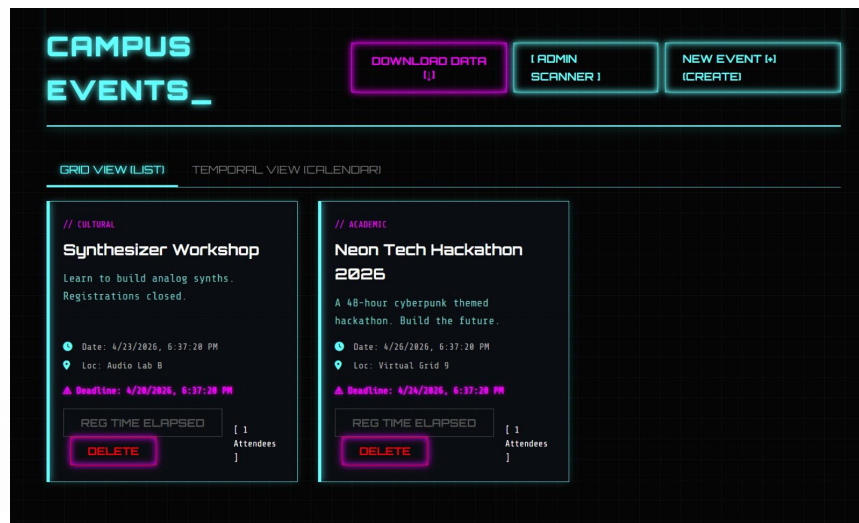


Figure 5.6: Registered Events

Figure 5.6: Describes the Registered Events module allows users to view and manage all the events they have enrolled in. It acts as a personalized section where users can track their participation and stay updated with event-related information. The module is implemented using React for the frontend and Spring Boot for backend services.

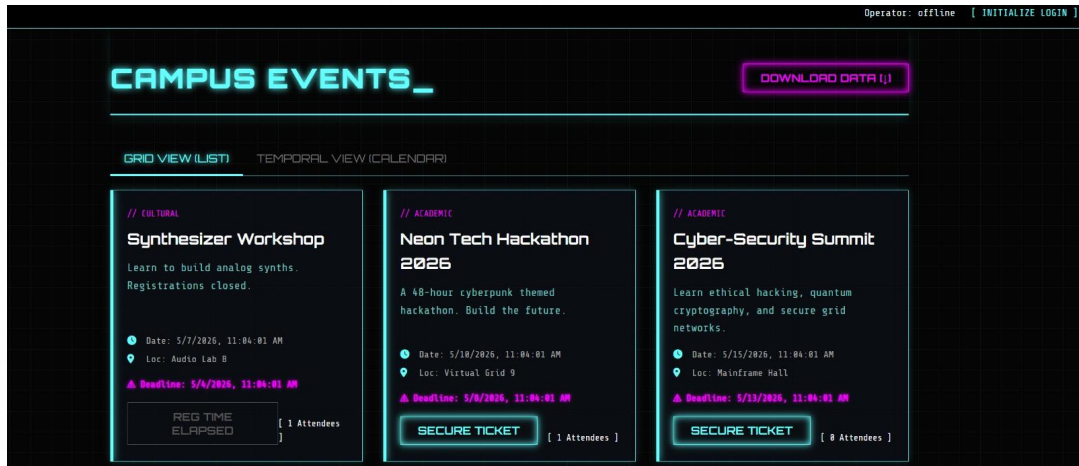


Figure 5.7: Profile Page

Figure 5.7: The Profile Page module lets users securely view and update their personal information. It is built with React and connects to Spring Boot backend services to keep user data accurate.

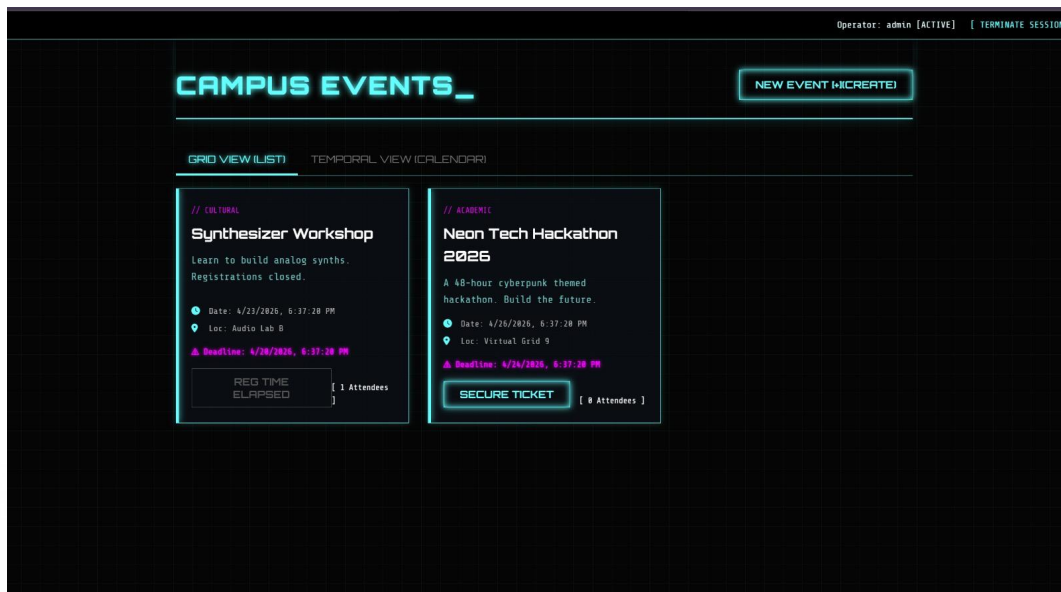


Figure 5.8: Signout Page

Figure 5.8: Describes the Sign Out module is responsible for securely terminating a user's session in the Smart Campus Event Management System. It ensures that once a user logs out, they can no

longer access protected resources without re-authentication. This module is implemented using React on the frontend and Spring Boot on the backend.

Discussion: The Smart Campus Event Management System proves to be an effective solution for managing campus events within educational institutions. Its modular and layered architecture ensures scalability, security, and ease of maintenance. The integration of key modules such as authentication, event management, registration, and feedback improves overall efficiency and reduces manual effort. The system enhances coordination between students and administrators while ensuring organized, reliable, and streamlined event management.

Table 5.1: Event Management System Module Comparison

Modules	Features
Landing Page	Displays upcoming events
Authentication	Login and registration for users
Event Listing	Shows campus event details
Event Registration	Registers users for events
Seat / Slot Selection	Selects available seats or time slots
QR Code / E-Pass	Generates digital entry passes
User Dashboard	Shows registered events
Admin Dashboard	Manages events and registrations
Notifications	Sends alerts and reminders
Reports	Shows event insights and analytics

Table 5.1 shows that the Smart Campus Event Management System is an effective solution for managing campus events in educational institutions. Its modular and layered architecture ensures scalability, security, and easy maintenance.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management System successfully provides an efficient solution for managing campus events and registrations. It reduces manual effort, minimizes errors, and ensures smooth event handling through a user-friendly interface. The system enhances overall event management by offering better control, accuracy, and convenience for both students and administrators.

6.2 Future Enhancements

While the current Smart Campus Event Management System provides a strong foundation for managing campus events, the following improvements can be considered for future development:

Online Payment Integration: Implement secure payment gateways to enable seamless event registration with multiple payment options.

QR Code-Based Check-in: Introduce QR code scanning for faster and more secure event entry and attendance tracking.

Advanced Analytics Dashboard: Provide detailed insights such as participation trends, event performance, and user engagement for better decision-making.

Email and SMS Notification System: Enable automated notifications for event registration, reminders, updates, and cancellations.

Role-Based Access Control: Expanding user roles (e.g., Super Admin, Organizer, Staff) to improve system security and provide better control over event management operations.

Seat Allocation Optimization: Enhance user roles such as Admin, Organizer, and Student for improved security and management.

Mobile Application Support: Develop a mobile app version to increase accessibility and user convenience.

Integration with External Problems: Connect with other systems like college portals or third-party services for extended functionality.

Chapter 7

Addressing Complex Engineering Problem and SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

WP1	Depth of Knowledge	Requires in-depth engineering	WK4, WK5	Knowledge of React JS, hooks, validation, UI design
WP2	Conflicting Requirements	Technical and non-technical constraints	WK4, WK5	Balances usability with ticket booking validation
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4	Component design, state handling, dynamic updates
WP4	Familiarity	Partially new problems	WK4	Real-time booking and ticket updates
WP5	Applicable Codes	Limited standard solutions	WK6	Custom booking and validation logic
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK6	Students, faculty, organizers
WP7	Interdependence	System-level integration	WK3, WK5	UI + state + validation integration

Table 7.1 The system requires strong engineering knowledge in React JS, including hooks, validation, and UI design, while balancing usability with ticket booking constraints. It involves analytical thinking for component design, state management, and real-time updates, often requiring custom logic due to limited standard solutions.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of Project to Complex Engineering Problem (CEP)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of technologies	React JS, HTML, CSS
EA2	Level of Interactions	Complex module interaction	Event display + booking + validation
EA3	Innovation	Modern technologies	Hooks, conditional rendering
EA4	Consequences	Impact on users	Simplifies booking process
EA5	Familiarity	Beyond standard solutions	Dynamic real-time system

Table 7.2 The system uses technologies like React JS, HTML, and CSS, enabling complex interactions such as event display, booking, and validation. It incorporates modern features like hooks and conditional rendering to enhance innovation, while simplifying the booking process for users through a dynamic, real-time system beyond standard solutions.

7.3 Sustainable Development Goals (SDG) Mapping

Table 7.3: SDG Mapping of the Project

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Supports academic events and learning
SDG 9	Industry, Innovation and Infrastructure	Promotes modern React-based systems
SDG 10	Reduced Inequalities	Accessible platform for all users
SDG 11	Sustainable Cities and Communities	Encourages organized and efficient event management

Table 7.3 The system supports SDG 4 by enhancing academic events and learning, aligns with SDG 9 through modern technologies, promotes inclusivity under SDG 10, and contributes to SDG 11 by ensuring organized and efficient campus event management.

Chapter 8

REFERENCES

- [1] React – UI development. <https://react.dev>
- [2] Spring Boot – Backend APIs. <https://spring.io/projects/spring-boot>
- [3] MySQL – Data storage. <https://www.mysql.com>
- [4] Axios – API requests. <https://axios-http.com>
- [5] Firebase – Authentication hosting. <https://firebase.google.com>
- [6] Bootstrap – UI styling. <https://getbootstrap.com>
- [7] Postman – API testing. <https://www.postman.com>
- [8] Node.js – Backend runtime. <https://nodejs.org>
- [9] Express.js – Web framework. <https://expressjs.com>
- [10] GitHub – Code repository. <https://github.com>